

# CSCE 46103/56103 Artificial Intelligence – Fall 2026

## Backtracking, Inference, and CSP Heuristics

Jungtaek Kim

`jungtaek.kim@uark.edu`

University of Arkansas

September 3, 2026

# An Impossible Partial Coloring

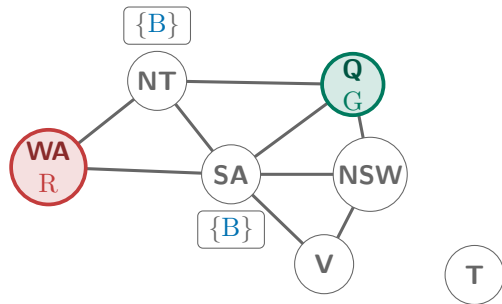
## A Partial Map Coloring

After assigning  $\boxed{\text{WA} = \text{R}}$  and  $\boxed{\text{Q} = \text{G}}$ , remove colors that conflict directly with those assignments. This leaves

$$D_{NT} = \{\text{B}\}, \quad D_{SA} = \{\text{B}\}.$$

Both variables are unassigned, and they share a border.

**Can this branch still become a valid coloring?**



# An Impossible Partial Coloring

## A Partial Map Coloring

After assigning  $\text{WA} = \text{R}$  and  $\text{Q} = \text{G}$ , remove colors that conflict directly with those assignments. This leaves

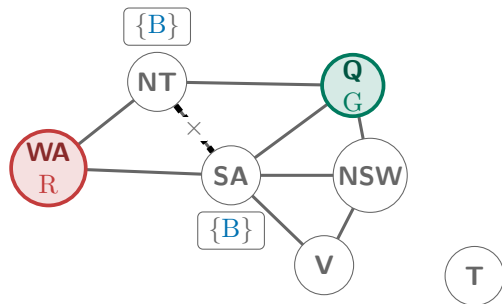
$$D_{NT} = \{\text{B}\}, \quad D_{SA} = \{\text{B}\}.$$

Both variables are unassigned, and they share a border.

**Can this branch still become a valid coloring?**

## A Contradiction before Another Choice

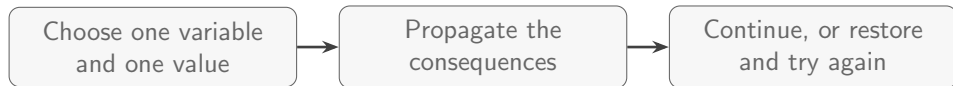
No. Both  $NT$  and  $SA$  would have to be blue, but neighboring regions must have different colors. The remaining question is how much reasoning a solver performs before its next choice.



# From Propagation to Search

## When Propagation Stops

Propagation removes values that cannot work. It may expose a contradiction, complete the assignment, or leave several legal choices. Search is needed only in the third case.

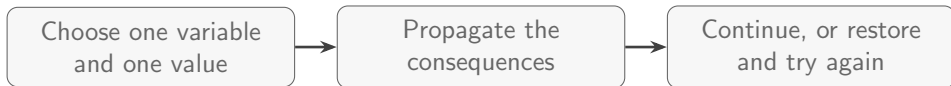


The next questions are **what to choose**, **how much to infer**, and **what to restore after failure**.

# From Propagation to Search

## When Propagation Stops

Propagation removes values that cannot work. It may expose a contradiction, complete the assignment, or leave several legal choices. Search is needed only in the third case.



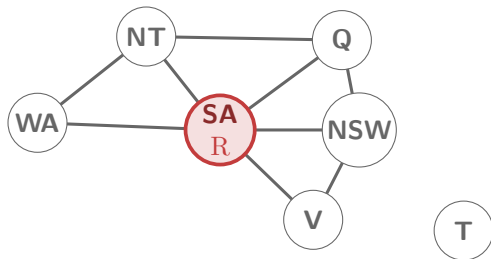
## Backtracking

When a choice leads to a contradiction, return to the state before that choice and try the next value. Repeat until a solution is found or every value has been tried.

The next questions are **what to choose**, **how much to infer**, and **what to restore after failure**.

# Backtracking: Try, Fail, Undo, Retry

Backtracking explores one choice at a time and restores the earlier state after failure.



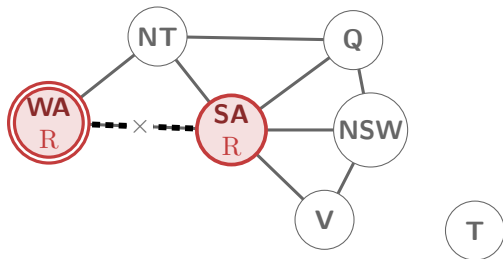
1. Assign  $SA = \text{R}$ .

Prediction

May the next choice  $WA = \text{R}$  be accepted?

# Backtracking: Try, Fail, Undo, Retry

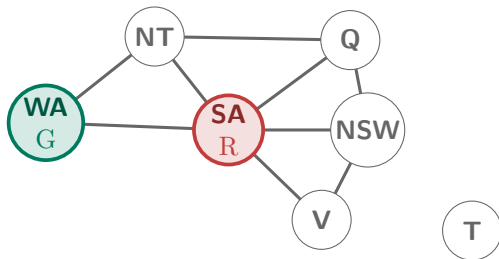
Backtracking explores one choice at a time and restores the earlier state after failure.



1. Assign  $SA = R$ .
2. Try  $WA = R$ : the edge  $WA \neq SA$  fails.

# Backtracking: Try, Fail, Undo, Retry

Backtracking explores one choice at a time and restores the earlier state after failure.



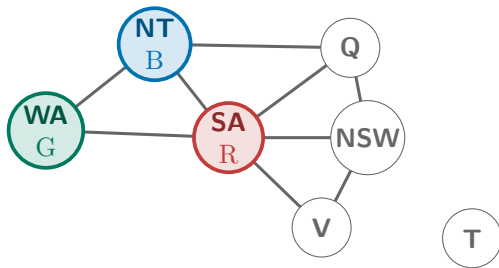
1. Assign  $SA = R$ .
2. Try  $WA = R$ : the edge  $WA \neq SA$  fails.
3. Undo  $WA = R$  and its consequences; then try  $WA = G$ .

## Restore Before Retrying

Undo the assignment and every domain deletion it caused. A sibling branch begins from the parent's state.

# Backtracking: Try, Fail, Undo, Retry

Backtracking explores one choice at a time and restores the earlier state after failure.



1. Assign  $SA = R$ .
2. Try  $WA = R$ : the edge  $WA \neq SA$  fails.
3. Undo  $WA = R$  and its consequences; then try  $WA = G$ .
4.  $NT$  cannot be  $R$  or  $G$ , so assign  $NT = B$ .

## Restore Before Retrying

Undo the assignment and every domain deletion it caused. A sibling branch begins from the parent's state.

# Backtracking Search Loop

Backtrack(assignment, domains)

1. If every variable is assigned, return the solution.
2. Choose one unassigned variable  $X$ .
3. For each value  $x$  in the current domain  $D_X$ :
  - 3.1 tentatively assign  $X = x$ ;
  - 3.2 check constraints and propagate;
  - 3.3 if no contradiction appears, recursively search the remaining variables;
  - 3.4 return any solution found; otherwise restore the parent state.
4. If no value works, return failure.

Two Possible Returns

- **Solution:** pass it upward.
- **Failure:** try the next value.

# Backtracking Search Loop

Backtrack(assignment, domains)

1. If every variable is assigned, return the solution.
2. Choose one unassigned variable  $X$ .
3. For each value  $x$  in the current domain  $D_X$ :
  - 3.1 tentatively assign  $X = x$ ;
  - 3.2 check constraints and propagate;
  - 3.3 if no contradiction appears, recursively search the remaining variables;
  - 3.4 return any solution found; otherwise restore the parent state.
4. If no value works, return failure.

Two Possible Returns

- **Solution:** pass it upward.
- **Failure:** try the next value.

## One Safe Habit

Before trying  $X = x$ , make sure the current assignments and domains can be restored exactly.

# One Variable per Search Level

## Two Orders, One Partial Assignment

*WA* **first**

$$\emptyset \longrightarrow \{WA = \text{R}\} \longrightarrow \{WA = \text{R}, Q = \text{G}\}$$

*Q* **first**

$$\emptyset \longrightarrow \{Q = \text{G}\} \longrightarrow \{WA = \text{R}, Q = \text{G}\}$$

Both orders reach the same partial assignment. Keeping both would duplicate the remaining search.

## Backtracking Organization

At each level, select exactly one unassigned variable and branch only on its values. Variable selection may change dynamically, but variable order is not an additional branch.

# One Variable per Search Level

## Two Orders, One Partial Assignment

*WA* first

$$\emptyset \longrightarrow \{WA = \text{R}\} \longrightarrow \{WA = \text{R}, Q = \text{G}\}$$

*Q* first

$$\emptyset \longrightarrow \{Q = \text{G}\} \longrightarrow \{WA = \text{R}, Q = \text{G}\}$$

Both orders reach the same partial assignment. Keeping both would duplicate the remaining search.

## Backtracking Organization

At each level, select exactly one unassigned variable and branch only on its values. Variable selection may change dynamically, but variable order is not an additional branch.

## Worst-Case Leaves without Pruning

If each of  $n$  variables has  $d$  values,

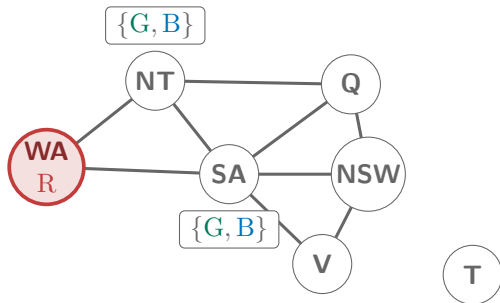
$$(nd)((n-1)d) \cdots d = n!d^n$$

versus

$$\underbrace{d \cdot d \cdots d}_{n \text{ levels}} = d^n.$$

# Variable-Ordering Decision

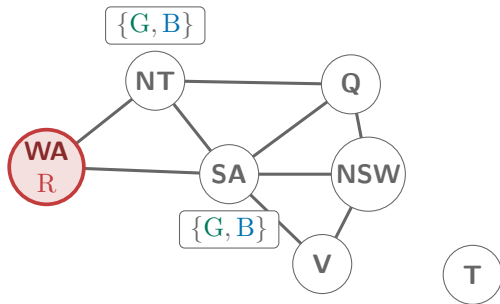
Here, current domains reflect conflicts with **assigned neighbors** only.



| Variable | Choices now | Unassigned neighbors |
|----------|-------------|----------------------|
| NT       | {G, B}      | 2                    |
| SA       | {G, B}      | 4                    |
| Q        | {R, G, B}   | 3                    |
| NSW      | {R, G, B}   | 3                    |
| V        | {R, G, B}   | 2                    |
| T        | {R, G, B}   | 0                    |

# Variable-Ordering Decision

Here, current domains reflect conflicts with **assigned neighbors** only.



| Variable | Choices now | Unassigned neighbors |
|----------|-------------|----------------------|
| NT       | {G, B}      | 2                    |
| SA       | {G, B}      | 4                    |
| Q        | {R, G, B}   | 3                    |
| NSW      | {R, G, B}   | 3                    |
| V        | {R, G, B}   | 2                    |
| T        | {R, G, B}   | 0                    |

## A Two-Step Answer

**Minimum remaining values (MRV)** ties *NT* and *SA* at two choices. The **degree** tie-breaker selects *SA*: it has four unassigned neighbors, versus two for *NT*.

# MRV and Degree Tie-Breaking

## Rules in Words

Choose the unassigned variable with the fewest current values. If several variables tie, choose the tied variable with the most still-unassigned neighbors.

## Australia Example

After  $WA = \text{R}$ ,  $NT$  and  $SA$  each have two choices, so MRV ties them. Degree then selects  $SA$  because it has four unassigned neighbors, while  $NT$  has two.

**MRV finds the tightest variable; degree breaks only an MRV tie.**

# MRV and Degree Tie-Breaking

## Rules in Words

Choose the unassigned variable with the fewest current values. If several variables tie, choose the tied variable with the most still-unassigned neighbors.

## MRV Notation

The variables with the fewest choices form

$$\mathcal{M} = \arg \min_{Y \text{ unassigned}} |D_Y|.$$

**Fail first:** test the fragile part now.

## Degree Notation

Within that tie set, select

$$X = \arg \max_{Y \in \mathcal{M}} \deg_{\text{unassigned}}(Y).$$

Degree never overrides a strict MRV preference.

# Value-Ordering Decision

## Least-Constraining Value (LCV)

Once a variable has been selected, try the value that removes the fewest choices from its unassigned neighbors. This is the **least-constraining value (LCV)** rule.

Suppose  $Q$  has been selected after  $WA = \text{R}$ ; this holds the variable fixed only to compare its values.

$$D_{NT} = D_{SA} = \{\text{G}, \text{B}\}, \quad D_{NSW} = \{\text{R}, \text{G}, \text{B}\}.$$

| Try            | Values removed from<br>$NT, SA, NSW$ | Total |
|----------------|--------------------------------------|-------|
| $Q = \text{R}$ |                                      |       |
| $Q = \text{G}$ |                                      |       |
| $Q = \text{B}$ |                                      |       |

## How to Read the Score

Each deleted neighbor value adds one. A lower total leaves more room for later choices.

# Value-Ordering Decision

## Least-Constraining Value (LCV)

Once a variable has been selected, try the value that removes the fewest choices from its unassigned neighbors. This is the **least-constraining value (LCV)** rule.

Suppose  $Q$  has been selected after  $WA = \text{R}$ ; this holds the variable fixed only to compare its values.

$$D_{NT} = D_{SA} = \{\text{G}, \text{B}\}, \quad D_{NSW} = \{\text{R}, \text{G}, \text{B}\}.$$

| Try            | Values removed from<br>$NT, SA, NSW$ | Total |
|----------------|--------------------------------------|-------|
| $Q = \text{R}$ | $0 + 0 + 1$                          | 1     |
| $Q = \text{G}$ | $1 + 1 + 1$                          | 3     |
| $Q = \text{B}$ | $1 + 1 + 1$                          | 3     |

## How to Read the Score

Each deleted neighbor value adds one. A lower total leaves more room for later choices.

## Result and Comparison Branch

LCV prefers  $Q = \text{R}$  (score 1). To compare inference methods, we deliberately follow the legal but harder choice  $Q = \text{G}$  (score 3).

# Ordering-Heuristic Summary

## Variables: Expose Trouble Early

- ▶ MRV finds the variable with the least room.
- ▶ Degree breaks an MRV tie using reach.
- ▶ A forced move or failure becomes visible sooner.

## Values: Keep Options Open

- ▶ LCV compares values for the chosen variable.
- ▶ It prefers the value that removes fewer choices.
- ▶ A solution may require less backtracking.

## A Fixed Branch for Comparing Inference

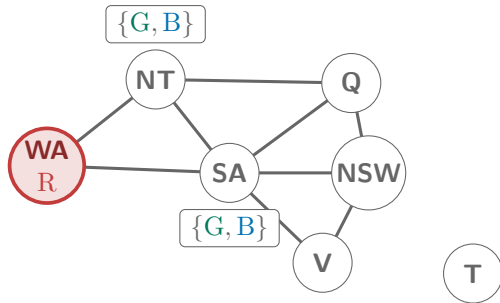
**WA=R**  $\longrightarrow$  **Q=G**  $\longrightarrow$  **NT=B**  $\longrightarrow$  **select** *SA*

This controlled trace does not follow the heuristic recommendation. Holding its decisions fixed lets us compare inference alone.

# Forward Checking on a Fixed Branch

## Forward Checking (FC)

After assigning  $X = x$ , inspect only the unassigned neighbors of  $X$ . Delete their values that conflict with  $x$ ; reject the branch if a neighbor domain becomes empty.



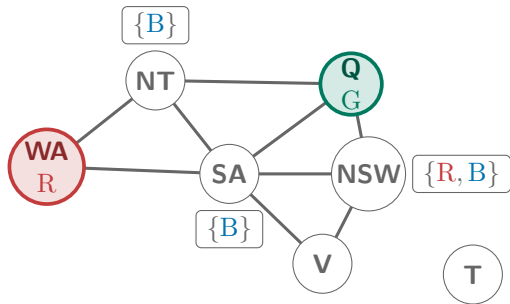
## One-Hop Updates

After  $WA = \mathbf{R}$ :  $D_{NT} = D_{SA} = \{\mathbf{G}, \mathbf{B}\}$ .

# Forward Checking on a Fixed Branch

## Forward Checking (FC)

After assigning  $X = x$ , inspect only the unassigned neighbors of  $X$ . Delete their values that conflict with  $x$ ; reject the branch if a neighbor domain becomes empty.



## One-Hop Updates

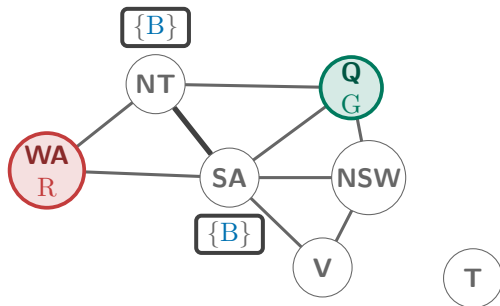
After  $WA = \text{R}$ :  $D_{NT} = D_{SA} = \{\text{G}, \text{B}\}$ .

After  $Q = \text{G}$ :  $D_{NT} = D_{SA} = \{\text{B}\}$  and  
 $D_{NSW} = \{\text{R}, \text{B}\}$ .

## What FC Concludes

Every updated domain is nonempty, so FC continues. It stops after this one-hop update and does not propagate from the new one-value domains.

# A Conflict beyond One Hop



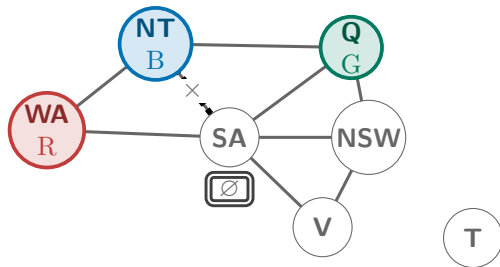
## What Forward Checking Sees

After  $Q = \text{G}$ ,  $D_{NT} = D_{SA} = \{\text{B}\}$ . Neither domain is empty.

## What It Misses

The constraint  $NT \neq SA$  cannot be satisfied, but it connects two unassigned variables. Forward checking does not test that edge.

# A Conflict beyond One Hop



## The Next Fixed Decision

Assign  $NT = B$ . Now  $SA$  is an unassigned neighbor of the newly assigned variable.

## Failure

Forward checking deletes  $B$  from  $D_{SA}$ :

$$D_{SA} : \{B\} \longrightarrow \emptyset.$$

The branch was impossible after  $Q = G$ , but FC needed  $NT = B$  to expose it.

# Maintaining Arc Consistency (MAC)

## AC-3 during Search

After each tentative assignment, MAC runs AC-3 starting with the affected arcs. Revision continues until the queue is empty or a domain becomes empty.

### Start the Queue

After propagating  $WA = \mathbf{R}$ ,  
 $D_{NT} = D_{SA} = \{\mathbf{G}, \mathbf{B}\}$ . Tentatively assign  
 $Q = \mathbf{G}$  and begin with

$$NT \rightarrow Q, \quad SA \rightarrow Q, \quad NSW \rightarrow Q.$$

On arc  $X \rightarrow Y$ , remove a value from  $D_X$  when  
 $D_Y$  contains no compatible partner.

### One Valid Queue Order

1. Revising arcs into  $Q$  gives

$$D_{NT} = D_{SA} = \{\mathbf{B}\}, \quad D_{NSW} = \{\mathbf{R}, \mathbf{B}\}.$$

# Maintaining Arc Consistency (MAC)

## AC-3 during Search

After each tentative assignment, MAC runs AC-3 starting with the affected arcs. Revision continues until the queue is empty or a domain becomes empty.

### Start the Queue

After propagating  $WA = \mathbf{R}$ ,  
 $D_{NT} = D_{SA} = \{\mathbf{G}, \mathbf{B}\}$ . Tentatively assign  
 $Q = \mathbf{G}$  and begin with

$$NT \rightarrow Q, \quad SA \rightarrow Q, \quad NSW \rightarrow Q.$$

On arc  $X \rightarrow Y$ , remove a value from  $D_X$  when  
 $D_Y$  contains no compatible partner.

### Failure before Another Decision

MAC rejects  $Q = \mathbf{G}$  before selecting  $NT$ . Restore  
the branch state.

### One Valid Queue Order

1. Revising arcs into  $Q$  gives

$$D_{NT} = D_{SA} = \{\mathbf{B}\}, \quad D_{NSW} = \{\mathbf{R}, \mathbf{B}\}.$$

2. Now  $D_{NT} = D_{SA} = \{\mathbf{B}\}$ . Revising  $SA \rightarrow NT$   
removes  $SA$ 's only value:

$$D_{SA} \longrightarrow \emptyset.$$

# Failure-Detection Timeline

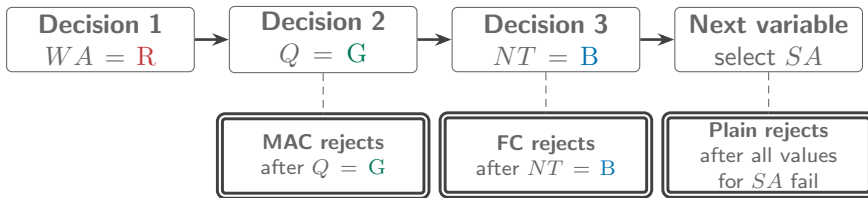
Continue with the same decision order whenever the branch has not already been rejected.



Predict where each method rejects the branch.

# Failure-Detection Timeline

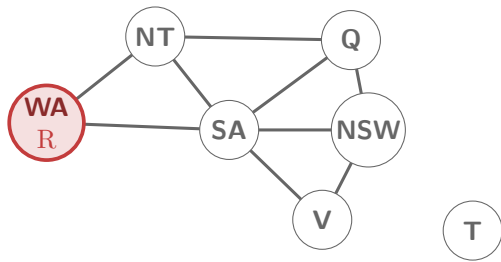
Continue with the same decision order whenever the branch has not already been rejected.



## Same Contradiction, Different Detection Times

MAC propagates between unassigned variables; FC waits until  $NT$  is assigned; plain backtracking exhausts  $SA$ 's values. Stronger inference does more work per node and may save work by rejecting the branch sooner.

# Guided Challenge: Choose, Order, Infer



After propagating  $WA = R$

FC and MAC give the same domains at this point.

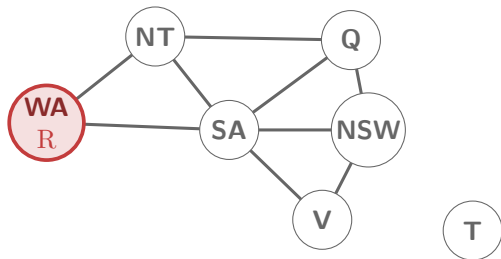
| Variable   | Current domain | Unassigned degree |
|------------|----------------|-------------------|
| <i>NT</i>  | {G, B}         | 2                 |
| <i>SA</i>  | {G, B}         | 4                 |
| <i>Q</i>   | {R, G, B}      | 3                 |
| <i>NSW</i> | {R, G, B}      | 3                 |
| <i>V</i>   | {R, G, B}      | 2                 |
| <i>T</i>   | {R, G, B}      | 0                 |

## Step 1: Choose a Variable

MRV finds a tie. Which variables tie, and which one does the degree tie-breaker select?

*Hint: find the smallest domains first.*

# Guided Challenge: Choose, Order, Infer



After propagating  $WA = R$

FC and MAC give the same domains at this point.

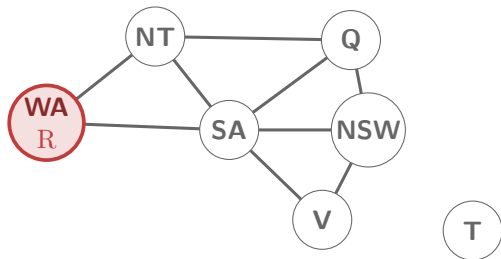
| Variable | Current domain | Unassigned degree |
|----------|----------------|-------------------|
| $NT$     | $\{G, B\}$     | 2                 |
| $SA$     | $\{G, B\}$     | 4                 |
| $Q$      | $\{R, G, B\}$  | 3                 |
| $NSW$    | $\{R, G, B\}$  | 3                 |
| $V$      | $\{R, G, B\}$  | 2                 |
| $T$      | $\{R, G, B\}$  | 0                 |

## Step 2: Order Its Values

Suppose  $SA$  is selected. Does LCV prefer  $G$  or  $B$ ?

*Hint: count how many neighboring values each choice removes.*

# Guided Challenge: Choose, Order, Infer



After propagating  $WA = R$

FC and MAC give the same domains at this point.

| Variable | Current domain | Unassigned degree |
|----------|----------------|-------------------|
| $NT$     | $\{G, B\}$     | 2                 |
| $SA$     | $\{G, B\}$     | 4                 |
| $Q$      | $\{R, G, B\}$  | 3                 |
| $NSW$    | $\{R, G, B\}$  | 3                 |
| $V$      | $\{R, G, B\}$  | 2                 |
| $T$      | $\{R, G, B\}$  | 0                 |

## Step 3: Infer

Try  $SA = G$ . What can MAC force before the solver makes another decision?

*Hint: follow each new one-value domain across its constraints.*

# Worked Answer: A Tie and a Forced Chain

## Choose a Variable

MRV ties  $NT$  and  $SA$  at two values. Degree selects  $SA$ , which constrains four unassigned neighbors.

Work from the domain sizes, then count unassigned neighbors.

# Worked Answer: A Tie and a Forced Chain

## Choose a Variable

MRV ties  $NT$  and  $SA$  at two values. Degree selects  $SA$ , which constrains four unassigned neighbors.

## Order Values

$G$  and  $B$  are symmetric here: each removes one color from the same four neighbors. LCV permits either.

With  $SA$  fixed, compare what  $G$  and  $B$  remove from its neighbors.

# Worked Answer: A Tie and a Forced Chain

## Choose a Variable

MRV ties  $NT$  and  $SA$  at two values. Degree selects  $SA$ , which constrains four unassigned neighbors.

## Order Values

$G$  and  $B$  are symmetric here: each removes one color from the same four neighbors. LCV permits either.

$$SA = G \rightarrow D_{NT} = \{B\}$$

$WA = R$  and  $SA = G$  leave  $NT$  only blue.

## Propagate

Choose  $SA = G$ . MAC turns local deletions into a forced chain across the mainland.

# Worked Answer: A Tie and a Forced Chain

## Choose a Variable

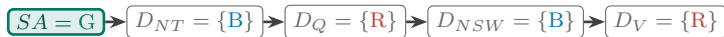
MRV ties  $NT$  and  $SA$  at two values. Degree selects  $SA$ , which constrains four unassigned neighbors.

## Order Values

$G$  and  $B$  are symmetric here: each removes one color from the same four neighbors. LCV permits either.

## Propagate

Choose  $SA = G$ . MAC turns local deletions into a forced chain across the mainland.



One possible AC-3 revision order is shown; every valid queue order produces the same final domains.  $T$  remains independent with domain  $\{R, G, B\}$ .

## Heuristic Ties Are Normal

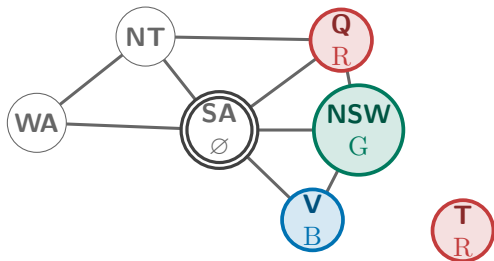
A tie is not a mistake: a heuristic may leave several equally good choices.

# Relevant Causes of Failure

Optional extension: use failure explanations after the basic search loop is comfortable.

## Plain Backtracking Branch (No Inference)

$Q = R \rightarrow NSW = G \rightarrow V = B \rightarrow T = R \rightarrow \text{test } SA's \text{ values}$



## Why *SA* Has No Color

**R** is blocked by  $Q = \text{R}$ ,  
**G** is blocked by  $NSW = \text{G}$ ,  
**B** is blocked by  $V = \text{B}$ .

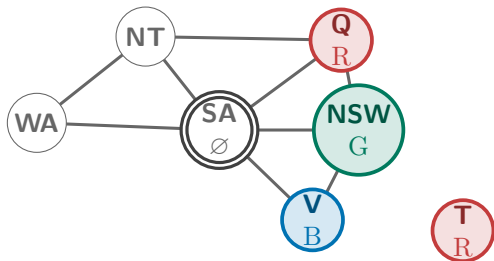
Chronological backtracking would undo *T* first. Could changing *T* help *SA*?

# Relevant Causes of Failure

Optional extension: use failure explanations after the basic search loop is comfortable.

## Plain Backtracking Branch (No Inference)

$Q=R \rightarrow NSW=G \rightarrow V=B \rightarrow T=R \rightarrow \text{test } SA's \text{ values}$



## Conflict Set and Backjump

A **conflict set** records earlier assignments responsible for the dead end:

$$\text{conf}(SA) = \{Q, NSW, V\}.$$

$T$  is absent because it has no constraint with  $SA$ . Simple backjumping skips  $T$  and returns to  $V$ , the most recent cause.

## Relevant rather than Recent

Backjumping follows the cause of failure, not merely the most recent assignment.

# Nogood Learning

## The Responsible Combination

Together,  $Q=R$ ,  $NSW=G$ , and  $V=B$  leave no color for  $SA$ . The unrelated assignment to  $T$  is not part of the explanation.

## Definition

A **nogood** is a partial assignment that cannot be extended to a solution.

# Nogood Learning

## The Responsible Combination

Together,  $Q=R$ ,  $NSW=G$ , and  $V=B$  leave no color for  $SA$ . The unrelated assignment to  $T$  is not part of the explanation.

## Definition

A **nogood** is a partial assignment that cannot be extended to a solution.

$$\text{Nogood: } N = \{Q = R, NSW = G, V = B\}$$

$$\text{Learned constraint: } \neg(Q = R \wedge NSW = G \wedge V = B)$$

If the nogood appears again, reject it immediately, regardless of unrelated choices or assignment order.

## Backjumping

Use the explanation to decide where to return now.

## Nogood Learning

Store the explanation to avoid the same failure later.

# Backtracking-Solver Summary

**Choose**  $\longrightarrow$  **Infer**  $\longrightarrow$  **Recover**  $\longrightarrow$  **Choose**

## Choose

MRV and degree select a variable. LCV orders its values.

## Infer

FC  $\longrightarrow$  MAC  
increasing inference strength

Stronger pruning usually costs more per node.

## Recover

On failure, restore the parent state and try the next value.

A complete assignment exits the loop as a solution.

Optional extension: explain failure to backjump or learn a nogood.

# Backtracking-Solver Summary

**Choose**  $\longrightarrow$  **Infer**  $\longrightarrow$  **Recover**  $\longrightarrow$  **Choose**

## Choose

MRV and degree select a variable. LCV orders its values.

## Infer

FC  $\longrightarrow$  MAC  
increasing inference strength

Stronger pruning usually costs more per node.

## Recover

On failure, restore the parent state and try the next value.

A complete assignment exits the loop as a solution.

Optional extension: explain failure to backjump or learn a nogood.

**Good ordering and inference keep bad branches small.**

# Thank You

Any questions?

# References

S. J. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020. URL <https://aima.cs.berkeley.edu/>.